DevOps Homework 5 - Winter 2026

Student: César Núñez
Date: 02/12/2026

Question 1: Compare and Contrast Deployment Strategies

Provide a comprehensive explanation of the following deployment strategies. For each strategy, include:

- How it works
- Advantages
- Disadvantages
- When to use it (use cases)

Deployment Strategy	How it works	Advantages	Disadvantages	Use cases	Source
Recreate deployment	This strategy involves terminating all existing instances of the old release before creating instances of the new release.	Simplicity: It is the simplest deployment strategy since it just runs one version of the software at a time.	Downtime: During the transition between old and new versions, there is a period where the application is not available for users.	Application with a unique instance running. In that case it will make sense to delete it and create it completely with the new version.	https://medium.com/@muppedaanvesh/rolling-update-recreate-deployment-strategies-in-kubernetes--327b59f27202
Rolling update	This strategy implements a gradual rollout of new releases, instead of updating all instances at once, it does it with a subset of instances.	Reduced downtime: guarantees that a subset of instances of the application are always running in a stable version. Incremental rollout: Allow for a gradual introduction of the new version which makes it easier to identify and solve issues without affecting all users.	Complex rollback: Need to revert updates on individual instances which can lead to inconsistancies if not properly managed. Longer deployment time: as this strategy occurs incrementally, the complete deployment process can take longer compared to other strategies.	Web application with multiple instances running. With rolling updates the application will continue to serve users even during updates	https://octopus.com/devops/software-deployments/rolling-deployment/ https://medium.com/@muppedaanvesh/rolling-update-recreate-deployment-strategies-in-kubernetes--327b59f27202

Deployment Strategy	How it works	Advantages	Disadvantages	Use cases	Source
Blue-Green deployment	Runs two identical production environments where the blue refers to the 'old' and green to the 'new' version of the program. During the deployment you can continue serving users in the blue environment while testing the green one. Then add progressively traffic into the green environment, fix issues and fully migrate when the green environment is properly working.	Less downtime: by having two identical environments users experience minimal or no downtime during updates. Immediate rollback: In case of a problem with the new release, you can redirect traffic into the stable environment.	Resource overhead: Maintaining 2 identical environment requires higher costs. Complexity in database changes: if the new release involves significant database schema changes, you have to plan ahead with a data synchronization and migration strategies to avoid inconsistencies. Security: running 2 environments increases the attack surface.	As can be really costly, this will fit for small or medium applications where the ammount of users still manageable to work with two independent production environments, such as an early stage mobile app.	https://octopus.com/devops/software-deployments/blue-green-deployment/

Deployment Strategy	How it works	Advantages	Disadvantages	Use cases	Source
Canary deployment	This strategy implements a rolling out deployment to a subset of users or serves, such that the first stage of deployment is on a small subset, test it and then roll it out progressively to the rest of the users/servers.	Risk mitigation: by deploying changes in small subset of servers, it's easy to find and address issues minimizing impact on users. Real-time feedback: provide immediate insights into the performance of new releases under real-world conditions. Cost-effectiveness: does not require a duplicated environment.	Complexity: requires sophisticated traffic management, monitoring and automated testing. Rollback complexity: relative to blue/green deployments, is more complex since requires to revert the version for a subset of users/servers.	New feature for an existing videogame or software in which you'd like to test it with a small subset of users.	https://octopus.com/devops/software-deployments/canary-deployment/

Deployment Strategy	How it works	Advantages	Disadvantages	Use cases	Source
A/B testing deployment	This strategy deploys simultaneously two versions of software to different user groups to compare some specific features and user response to those changes. This allow to make data-driven decisions about which versions should be fully deployed.	Optimizing User Experience: Allow to experiment with small changes that might impact users in a small and controlled setting. Data-Informed Decisions: reduce subjective opinions by providing concrete data for technical decisions by determining which variants of the software perform better	Potential False Positives: as it involves statistical analysis, there is a risk of incorrectly identify a variation as successful when it's actually due to chance. External factors: other factors might affect the results of the A/B testing and therefore the timing of the testing is important.	New payment interface for a online store	https://octopus.com/devops/software-deployments/#8-ab-testing https://www.enov8.com/blog/a-b-testing-the-good-the-bad/

Question 2: Rolling Update Implementation

Create a complete Kubernetes deployment for a sample web application using minikube.

Requirements:

- Use nginx image (or any web server of your choice)
- Configure 3 replicas
- Implement rolling update strategy with:
  - maxSurge: 1
  - maxUnavailable: 1
- Include health checks:
  - Readiness probe
  - Liveness probe

Deliverables:

- `rolling-update-deployment.yaml` - Complete deployment manifest: available here.
- `rolling-update-service.yaml` - Service manifest to expose the deployment: available here.
- `update-demo.sh` - Shell script demonstrating how to update the deployment to a new version: available here.
- `screenshots/` folder containing (available here):
  - `01-initial-deployment.png` - get deployments/pods showing initial state:

```
======================================================================
Deployment application from rolling-update-deployment.yaml...
deployment.apps/h5-q2 created

Initial state of deployment...
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
h5-q2   0/3     3            0           5s

NAME                     READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
h5-q2-6595488c46-76947   1/1     Running   0          11s   10.244.0.111   minikube   <none>           <none>
h5-q2-6595488c46-h6s58   1/1     Running   0          11s   10.244.0.110   minikube   <none>           <none>
h5-q2-6595488c46-zj7l4   1/1     Running   0          11s   10.244.0.109   minikube   <none>           <none>

Exposing the app...
service/h5-q2 created

Current image running: nginx:trixie-perl
```

  - `02-update-in-progress.png` - rollout status during update:

```
==============================================================================
Initializing Rolling update to nginx:stable-alpine-perl...
deployment.apps/h5-q2 image updated
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
deployment "h5-q2" successfully rolled out

==============================================================================
```

  - `03-update-completed.png` - get pods showing successful update:

```
======================================================================
Rolling update completed...
NAME                     READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
h5-q2-565c9777db-785tt   1/1     Running   0          7s    10.244.0.123   minikube   <none>           <none>
h5-q2-565c9777db-8l8cp   1/1     Running   0          15s   10.244.0.122   minikube   <none>           <none>
h5-q2-565c9777db-ld6dq   1/1     Running   0          15s   10.244.0.121   minikube   <none>           <none>
Current image running: nginx:stable-alpine-perl

======================================================================
```

- `04-rollback-executed.png` - rollout undo command and result:

```
==========================================================================
Initializing Rollback
deployment.apps/h5-q2 rolled back
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "h5-q2" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
Waiting for deployment "h5-q2" rollout to finish: 2 of 3 updated replicas are available...
deployment "h5-q2" successfully rolled out
Rollback completed...
Current image running: nginx:trixie-perl
```

- `05-service-endpoint.png` - curl or browser showing the service is accessible:

```
canun:q2 (main)$ curl http://127.0.0.1:44023
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Question 3: Blue-Green Deployment

Design and implement a complete blue-green deployment strategy for a web application using minikube. Requirements:

- Create separate deployments for blue and green environments
- Each environment should have at least 2 replicas
- Implement a Service that can switch traffic between blue and green
- Document the complete switching process

Deliverables:

- `blue-deployment.yaml` - Blue environment deployment: available here.
- `green-deployment.yaml` - Green environment deployment: available here.
- `blue-green-service.yaml` - Service configuration: available here.
- `switch-traffic.sh` - Script to switch traffic from blue to green: available here.
- `rollback.sh` - Script to rollback from green to blue: available here.
- `screenshots/` folder containing (available here):
  - `01-blue-deployment.png` - showing blue deployment running:

```
==========================================================================
Creating the Blue deployment from blue-deployment.yaml...
deployment.apps/h5-q3-blue created

Initial state of blue deployment...
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
h5-q3-blue    0/2     2            0           5s

NAME                           READY   STATUS    RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
h5-q3-blue-7894cdf66d-5rhsq    1/1     Running   0          10s   10.244.0.180   minikube   <none>           <none>
h5-q3-blue-7894cdf66d-7ghwg    1/1     Running   0          10s   10.244.0.179   minikube   <none>           <none>
```

  - `02-green-deployment.png` - showing both blue and green running:

```
==========================================================================
Initializing Blue-Green Deployment...
Creating the Green deployment from green-deployment.yaml...
deployment.apps/h5-q3-green created

==========================================================================
Initial state of deployment after the creation of Green environment...
NAME                           READY   STATUS              RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
h5-q3-blue-7894cdf66d-9z5kh    1/1     Running             0          11s   10.244.0.187   minikube   <none>           <none>
h5-q3-blue-7894cdf66d-f99mw    1/1     Running             0          11s   10.244.0.188   minikube   <none>           <none>
h5-q3-green-8b6b4d487-g8crx    0/1     ContainerCreating   0          0s    <none>         minikube   <none>           <none>
h5-q3-green-8b6b4d487-gws7s    0/1     ContainerCreating   0          0s    <none>         minikube   <none>           <none>

==========================================================================
Final state of deployment after the creation of Green environment...
NAME                           READY   STATUS    RESTARTS   AGE   IP             NODE       NOMINATED NODE   READINESS GATES
h5-q3-blue-7894cdf66d-9z5kh    1/1     Running   0          20s   10.244.0.187   minikube   <none>           <none>
h5-q3-blue-7894cdf66d-f99mw    1/1     Running   0          20s   10.244.0.188   minikube   <none>           <none>
h5-q3-green-8b6b4d487-g8crx    1/1     Running   0          9s    10.244.0.190   minikube   <none>           <none>
h5-q3-green-8b6b4d487-gws7s    1/1     Running   0          9s    10.244.0.189   minikube   <none>           <none>
==========================================================================
```

- `03-service-pointing-blue.png` - describe service showing blue selector:

```
Exposing the app...
service/h5-q3-service configured

Current image running: nginx:1.28-alpine-perl
==============================================================================
Describing service...
Name:                     h5-q3-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=nginx,color=blue,version=v1.0
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.32.115
IPs:                      10.96.32.115
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
Endpoints:                10.244.0.179:80,10.244.0.180:80
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>

==============================================================================
```

- `04-switch-to-green.png` - commands switching service to green:

```
==================================================================
Migrating service to Green environment...
service/h5-q3-service patched
NAME                        READY   STATUS    RESTARTS   AGE
h5-q3-green-8b6b4d487-g8crx   1/1     Running   0          10s
h5-q3-green-8b6b4d487-gws7s   1/1     Running   0          10s

==================================================================
Green is Ready with 2 pod running. Safe to clean up blue pods.
deployment.apps/h5-q3-blue scaled
Showing all available pods
NAME                        READY   STATUS        RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
h5-q3-blue-7894cdf66d-9z5kh   1/1     Terminating   0          21s   10.244.0.187   minikube   <none>           <none>
h5-q3-blue-7894cdf66d-f99mw   1/1     Terminating   0          21s   10.244.0.188   minikube   <none>           <none>
h5-q3-green-8b6b4d487-g8crx   1/1     Running       0          10s   10.244.0.190   minikube   <none>           <none>
h5-q3-green-8b6b4d487-gws7s   1/1     Running       0          10s   10.244.0.189   minikube   <none>           <none>
```

- `05-service-pointing-green.png` - describe service showing green selector:

```
==============================================================================
Describing service...
Name:                     h5-q3-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=nginx,color=green,version=v1.1
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.32.115
IPs:                      10.96.32.115
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
Endpoints:                10.244.0.181:80,10.244.0.182:80
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
```

- `06-rollback-to-blue.png` - Execution of rollback script and verification:

```
canun:q3 (main)$ ./rollback.sh v1.0
==============================================================================
Restoring 2 replicas at blue environment...
deployment.apps/h5-q3-blue scaled
Rolling back service to blue pods
service/h5-q3-service patched
Final state of deployment after the rollback...
NAME                          READY   STATUS    RESTARTS   AGE
h5-q3-blue-7894cdf66d-2wkt8   0/1     Running   0          6s
h5-q3-blue-7894cdf66d-4qqhk   0/1     Running   0          6s
h5-q3-green-8b6b4d487-8gzq2   1/1     Running   0          31s
h5-q3-green-8b6b4d487-9p6xm   1/1     Running   0          31s
Describing service...
Name:                     h5-q3-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=nginx,color=blue,version=v1.0
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.108.43.246
IPs:                      10.108.43.246
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
Endpoints:
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Question 4: Canary Deployment

Implement a canary deployment strategy that gradually rolls out a new version using minikube.

Requirements:

- Deploy a stable version (v1) and a canary version (v2)
- Route approximately 20% of traffic to canary (v2)
- Route approximately 80% of traffic to stable (v1)
- Provide a method to gradually increase traffic to canary

Deliverables:

- `stable-deployment.yaml` - Stable version deployment: available here
- `canary-deployment.yaml` - Canary version deployment: available here
- `stable-service.yaml` - Service for stable deployment: available here
- `canary-service.yaml` - Service for canary deployment: available here
- `canary-ingress.yaml` - Ingress configuration for traffic splitting: available here
- `canary-promotion.sh` - Script to gradually increase traffic to canary: available here
- `screenshots/` folder containing (available here):
  - `01-stable-deployment.png` - showing stable deployment running:

```
========================================================================
Creating the Stable deployment and service...
deployment.apps/h5-q4-stable created
service/stable-service created
Waiting for deployment "h5-q4-stable" rollout to finish: 0 of 4 updated replicas are available...
Waiting for deployment "h5-q4-stable" rollout to finish: 1 of 4 updated replicas are available...
Waiting for deployment "h5-q4-stable" rollout to finish: 2 of 4 updated replicas are available...
Waiting for deployment "h5-q4-stable" rollout to finish: 3 of 4 updated replicas are available...
deployment "h5-q4-stable" successfully rolled out

========================================================================
Initial state of stable deployment...
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
h5-q4-stable   4/4     4            4           6s

NAME                            READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
h5-q4-stable-56f468488f-55x2d   1/1     Running   0          6s    10.244.1.19   minikube   <none>           <none>
h5-q4-stable-56f468488f-8lv74   1/1     Running   0          6s    10.244.1.16   minikube   <none>           <none>
h5-q4-stable-56f468488f-gdlrn   1/1     Running   0          6s    10.244.1.18   minikube   <none>           <none>
h5-q4-stable-56f468488f-nk8l5   1/1     Running   0          6s    10.244.1.17   minikube   <none>           <none>
```

  - `02-canary-deployment.png` - showing both stable and canary running:

```
========================================================================
Creating the canary deployment, service and ingress manifests...
deployment.apps/h5-q4-canary created
service/canary-service created
ingress.networking.k8s.io/stable-ingress created
ingress.networking.k8s.io/canary-ingress created
Waiting for deployment "h5-q4-canary" rollout to finish: 0 of 1 updated replicas are available...
deployment "h5-q4-canary" successfully rolled out

========================================================================
Stable + canary running:
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
h5-q4-canary   1/1     1            1           0s
h5-q4-stable   4/4     4            4           7s
NAME                            READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
h5-q4-canary-5f54f95d5c-wcwnd   1/1     Running   0          1s    10.244.1.20   minikube   <none>           <none>
h5-q4-stable-56f468488f-55x2d   1/1     Running   0          8s    10.244.1.19   minikube   <none>           <none>
h5-q4-stable-56f468488f-8lv74   1/1     Running   0          8s    10.244.1.16   minikube   <none>           <none>
h5-q4-stable-56f468488f-gdlrn   1/1     Running   0          8s    10.244.1.18   minikube   <none>           <none>
h5-q4-stable-56f468488f-nk8l5   1/1     Running   0          8s    10.244.1.17   minikube   <none>           <none>
```

- `03-traffic-split-20-80.png` - describe showing initial traffic split:

```
==========================================================================
Showing initial canary split: 20/80...
ingress.networking.k8s.io/canary-ingress annotated
Name:             canary-ingress
Labels:           <none>
Namespace:        default
Address:
Ingress Class:    <none>
Default backend:  <default>
Rules:
  Host         Path  Backends
  ----         ----  --------
  h5-q4.local
               /   canary-service:80 (10.244.1.20:5678)
Annotations:   nginx.ingress.kubernetes.io/canary: true
               nginx.ingress.kubernetes.io/canary-weight: 20
               nginx.ingress.kubernetes.io/ssl-redirect: false
Events:
  Type    Reason  Age   From                      Message
  ----    ------  ----  ----                      -------
  Normal  Sync    1s    nginx-ingress-controller  Scheduled for sync
```

- `04-traffic-test.png` - Multiple curl requests showing ~20% going to canary:

```
==========================================================================
Showing initial canary split: 20/80...
Showing sample responses...
Request  1 → stable v1
Request  2 → canary v2
Request  3 → stable v1
Request  4 → stable v1
Request  5 → stable v1
Request  6 → stable v1
Request  7 → stable v1
Request  8 → canary v2
Request  9 → stable v1
Request 10 → stable v1

Showing summary results...
canary v2    : 10 (20%)
stable v1    : 40 (80%)
```

- `05-promotion-50-50.png` - After promotion showing 50/50 traffic split:

```
==========================================================================
Promoting canary to 50%...
ingress.networking.k8s.io/canary-ingress annotated
Name:             canary-ingress
Labels:           <none>
Namespace:        default
Address:
Ingress Class:    <none>
Default backend:  <default>
Rules:
  Host         Path  Backends
  ----         ----  --------
  h5-q4.local
               /   canary-service:80 (10.244.1.35:5678)
Annotations:   nginx.ingress.kubernetes.io/canary: true
               nginx.ingress.kubernetes.io/canary-weight: 50
               nginx.ingress.kubernetes.io/ssl-redirect: false
Events:
  Type    Reason  Age              From                      Message
  ----    ------  ----             ----                      -------
  Normal  Sync    1s (x2 over 6s)  nginx-ingress-controller  Scheduled for sync
Showing sample responses...
Request  1 → stable v1
Request  2 → stable v1
Request  3 → canary v2
Request  4 → canary v2
Request  5 → stable v1
Request  6 → canary v2
Request  7 → canary v2
Request  8 → stable v1
Request  9 → stable v1
Request 10 → canary v2

Showing summary results...
canary v2    : 30 (60%)
stable v1    : 20 (40%)
```

- `06-promotion-100.png` - Final state with 100% traffic to canary:

```
==========================================================================
Promoting canary to 100%...
ingress.networking.k8s.io/canary-ingress annotated
Name:             canary-ingress
Labels:           <none>
Namespace:        default
Address:
Ingress Class:    <none>
Default backend:  <default>
Rules:
  Host         Path  Backends
  ----         ----  --------
  h5-q4.local
               /   canary-service:80 (10.244.1.35:5678)
Annotations:   nginx.ingress.kubernetes.io/canary: true
               nginx.ingress.kubernetes.io/canary-weight: 100
               nginx.ingress.kubernetes.io/ssl-redirect: false
Events:
  Type    Reason  Age              From                      Message
  ----    ------  ----             ----                      -------
  Normal  Sync    0s (x3 over 10s) nginx-ingress-controller  Scheduled for sync
Showing sample responses...
Request  1 → canary v2
Request  2 → canary v2
Request  3 → canary v2
Request  4 → canary v2
Request  5 → canary v2
Request  6 → canary v2
Request  7 → canary v2
Request  8 → canary v2
Request  9 → canary v2
Request 10 → canary v2

Showing summary results...
canary v2    : 50 (100%)
```